Lecture 9: Constrained Optimization and PCA

Math for Data Science · 2026

Jan Gieseler

2026-11-03

Agenda

- Part I: Constraint Optimization
- Part II: Optimization in Matrix-Vector Land
- Part III: Principal Component Analysis (PCA)
- Part IV: Further Learning - Optimization Algorithms
- Appendix

Part I: Constraint Optimization

Recap: Optimization with a single variable

For $f : \mathbb{R} \to \mathbb{R}$, lecture 4's recipe was:

1. Solve $f'(x) = 0$ - the critical points
2. Check $f''(x)$ - positive is a minimum, negative a maximum, 0 fall back to first derivative test

Today we add

- more variables $f : \mathbb{R}^k \to \mathbb{R}$
- constraints
- step one becomes a system of equations that we need to solve
- f'' in step two turns into a matrix (the Hessian)

Example 1 - Optimizing with a Constraint

Suppose we want our solution to be subject to some constraint. For instance, we want to minimize:

$$\min_{x,y} \; x + y$$

and we want the solution to live on the outside of a circle, given by $x^2 + y^2 = 1$. Then, we would:

1. Set up our constraint as a function $g = 0$: $x^2 + y^2 - 1 = 0$
2. Form the Lagrangian

$$\mathcal{L} = f(x, y) + \lambda g(x, y) = x + y + \lambda(x^2 + y^2 - 1)$$

3. Set every partial derivative of $\mathcal{L}$ to zero forming our first-order conditions
4. Solve the system of equations for x and y.

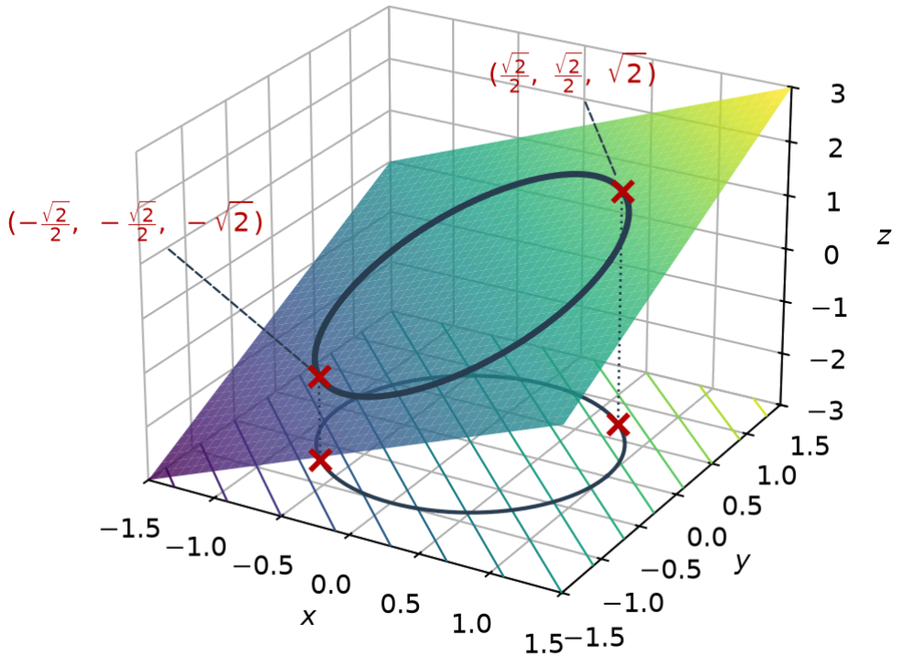


Figure after Wikipedia: Lagrange multiplier.

Example 1, Solved

$$\mathcal{L}(x, y, \lambda) = x + y + \lambda(x^2 + y^2 - 1)$$

$$\begin{aligned} \partial\mathcal{L}/\partial x = 1 + 2\lambda x = 0 \quad &\Rightarrow \quad x = -\tfrac{1}{2\lambda} \\ \partial\mathcal{L}/\partial y = 1 + 2\lambda y = 0 \quad &\Rightarrow \quad y = -\tfrac{1}{2\lambda} \\ \partial\mathcal{L}/\partial \lambda = x^2 + y^2 - 1 = 0 \quad & \end{aligned}$$

Substitute the first two into the third: $2 \cdot \frac{1}{4\lambda^2} = 1$, so $\lambda = \pm\frac{1}{\sqrt{2}}$.

$$\lambda = \tfrac{1}{\sqrt{2}} : \ (x^*, y^*) = \left(-\tfrac{1}{\sqrt{2}}, -\tfrac{1}{\sqrt{2}}\right), \ \mathcal{L}(x^*, y^*) = f^* = -\sqrt{2}$$

$$\lambda = -\tfrac{1}{\sqrt{2}} : \ (x^*, y^*) = \left(\tfrac{1}{\sqrt{2}}, \tfrac{1}{\sqrt{2}}\right), \ f^* = +\sqrt{2}$$

λ^* is commonly refered to: “Shadow price”. It is the money gained by loosening the constraint by a single dollar, or conversely the price of strengthening the constraint by one dollar. See Interpretation of Lagrange multipliers (Khan Acadamy)

For this example also see See p.84 Hertzmann, Fleet and Brubaker, CSC 411 Course Notes (University of Toronto, 2015)

Example 2 - MLE for the Multinomial

N observations fall into K categories; N_k land in category k, with probability p_k.

1. Write the likelihood

$$\mathcal{L}_0 = \left(\frac{N!}{N_1! N_2! \cdots N_K!} \right) \prod_{k=1}^{K} p_k^{N_k}$$

2. Take the log

$$\ell = \log\left(\frac{N!}{N_1! \cdots N_K!} \right) + \sum_{k=1}^{K} N_k \log p_k$$

3. Add the contraint $\sum_{k=1}^{K} p_k = 1$ (probabilities must sum to one) and write down the Lagrangian

$$\mathcal{L} = \log\left(\frac{N!}{N_1! \cdots N_K!} \right) + \sum_{k=1}^{K} N_k \log p_k + \lambda \left(\sum_{k=1}^{K} p_k - 1 \right)$$

MLE for the Multinomial with $K = 3$

$$\mathcal{L} = N_1 \log p_1 + N_2 \log p_2 + N_3 \log p_3 + \lambda(p_1 + p_2 + p_3 - 1)$$

$$\begin{aligned}
\partial\mathcal{L}/\partial p_1 &= \frac{N_1}{p_1} + \lambda = 0 \quad &\Rightarrow \quad \hat{p}_1 &= -\frac{N_1}{\lambda} \\
\partial\mathcal{L}/\partial p_2 &= \frac{N_2}{p_2} + \lambda = 0 \quad &\Rightarrow \quad \hat{p}_2 &= -\frac{N_2}{\lambda} \\
\partial\mathcal{L}/\partial p_3 &= \frac{N_3}{p_3} + \lambda = 0 \quad &\Rightarrow \quad \hat{p}_3 &= -\frac{N_3}{\lambda} \\
\partial\mathcal{L}/\partial \lambda &= p_1 + p_2 + p_3 - 1 = 0
\end{aligned}$$

Substitute back, we get $\lambda^* = -N$ and:

$$\hat{p}_k^* = \frac{N_k}{N_1 + N_2 + N_3} = \frac{N_k}{N}$$

The MLE of the multinomial is the observed proportion.

Probably you would have guessed it right. Now you know how to derive it.

Part II: Optimization in Matrix-Vector Land

The Gradient

Let f be a function that takes some input in $\mathbb{R}^m$ and outputs some real value $\in \mathbb{R}$. For instance, let

$$f(\mathbf{z}) = \mathbf{z}^T \mathbf{z}$$

for some vector $\mathbf{z}$ of length m.

Then, we can write the gradient of $f(\mathbf{z})$ as the vector of first derivatives of $\mathbf{z}$ with respect to $\mathbf{z}$:

$$\nabla_{\mathbf{z}} f(\mathbf{z}) = \begin{bmatrix} \partial f / \partial z_1 \\ \partial f / \partial z_2 \\ \vdots \\ \partial f / \partial z_m \end{bmatrix} \in \mathbb{R}^m$$

The gradient is a vector of the same shape as the input, and it points in the direction of steepest increase.

The nabla operator ∇ f is a shorthand for the vector $(\partial_x f, \partial_y f, \ldots)$

The Gradient with Respect to a Matrix

Similarly, f can be a function that takes some input in $\mathbb{R}^{m \times n}$ and outputs some real value $\in \mathbb{R}$. Then, the gradient will be a matrix $\in \mathbb{R}^{m \times n}$:

$$(\nabla_{\mathbf{A}} f(\mathbf{A}))_{ij} = \frac{\partial f(\mathbf{A})}{\partial A_{ij}}$$

$$\nabla_{\mathbf{A}} f(\mathbf{A}) = \begin{bmatrix} \frac{\partial f}{\partial A_{11}} & \cdots & \frac{\partial f}{\partial A_{1n}} \\ \vdots & \ddots & \vdots \\ \frac{\partial f}{\partial A_{m1}} & \cdots & \frac{\partial f}{\partial A_{mn}} \end{bmatrix}$$

We Can Perform Constrained Optimization Using the Gradient

The first-order conditions were: every partial derivative of $\mathcal{L}$ is zero. In vector notation that is one equation.

$$\nabla \mathcal{L}(x, y, \dots, \lambda) = \mathbf{0}$$

From $\mathcal{L}(x, y, \dots, \lambda) = f(x, y, \dots) + \lambda\, g(x, y, \dots)$, we find

1. $\nabla f = -\lambda \nabla g$, i.e. at the optimum, the level curve of f is tangent to the constraint curve*.
2. $g(x, y, \dots) = 0$, i.e. the constraint reappears as a first-order condition.

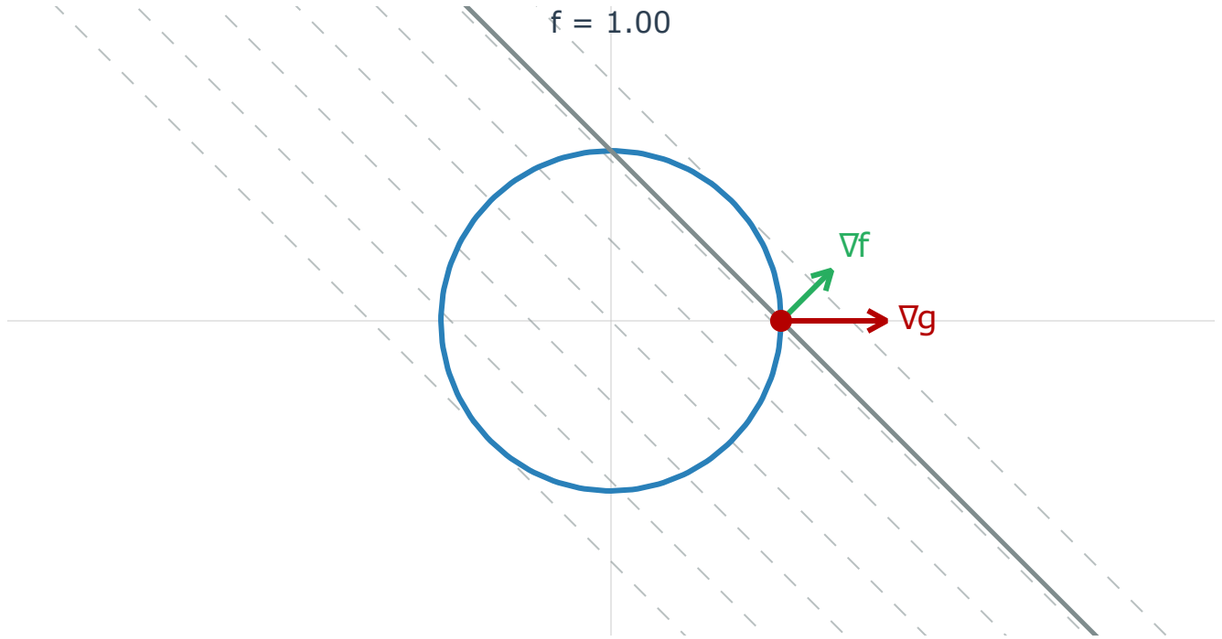
*If the two were not parallel, part of ∇f would point along the constraint. We could move that way and do better.

Example 1 in vector notation

$$\mathcal{L}(x, y, \lambda) = x + y + \lambda(x^2 + y^2 - 1)$$

$$\nabla\mathcal{L}(x, y, \lambda) = \begin{bmatrix} 1 + 2\lambda x \\ 1 + 2\lambda y \\ x^2 + y^2 - 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Visualizing the gradients for example 1



Green is ∇f (the same everywhere in this example); red is ∇g (the outward normal).

Minimum or Maximum? Introducing the Hessian

How do we know if we found a minimum or maximum? The matrix of second derivatives is called the Hessian matrix:

$$\mathbf{H} = \begin{bmatrix} \frac{\partial^2 \mathcal{L}}{\partial x^2} & \frac{\partial^2 \mathcal{L}}{\partial x \partial y} & \frac{\partial^2 \mathcal{L}}{\partial x \partial \lambda} \\ \frac{\partial^2 \mathcal{L}}{\partial y \partial x} & \frac{\partial^2 \mathcal{L}}{\partial y^2} & \frac{\partial^2 \mathcal{L}}{\partial y \partial \lambda} \\ \frac{\partial^2 \mathcal{L}}{\partial \lambda \partial x} & \frac{\partial^2 \mathcal{L}}{\partial \lambda \partial y} & \frac{\partial^2 \mathcal{L}}{\partial \lambda^2} \end{bmatrix}$$

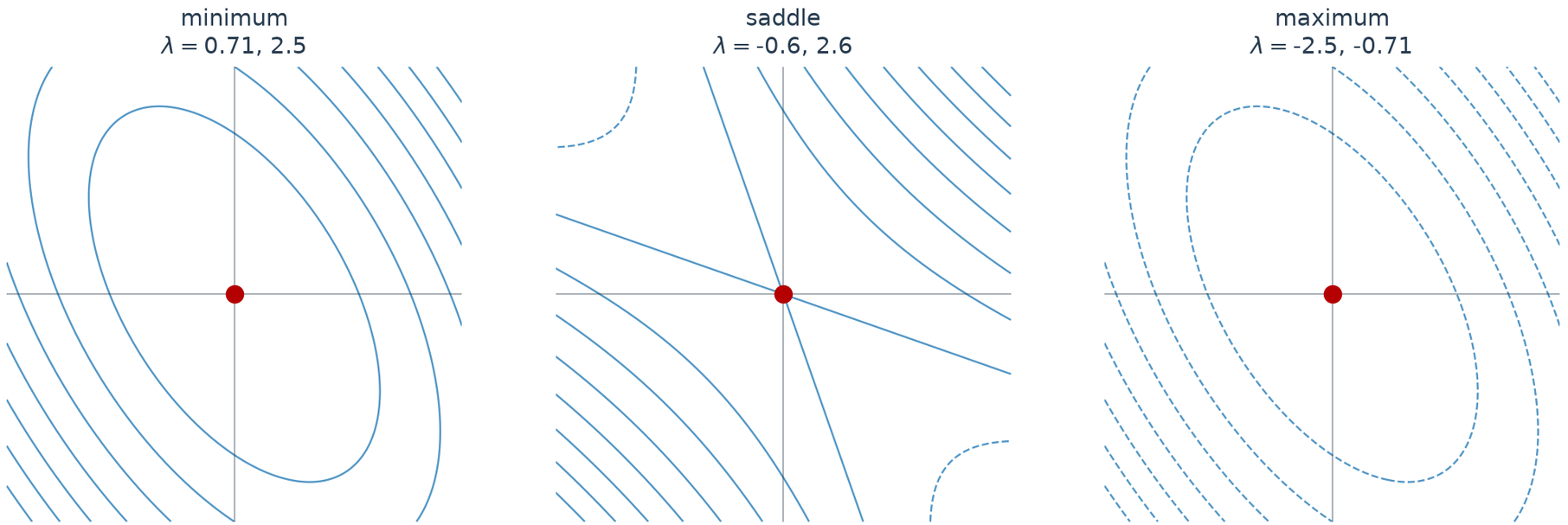
It is symmetric: differentiating in x then y gives the same thing as y then x.

A symmetric matrix has real eigenvalues and orthogonal eigenvectors.

At a critical point, eigenvalues are:

- all positive → a local minimum (curves up in every direction)
- all negative → a local maximum
- mixed → a saddle point

Contour plots for the three cases



Contours of $\frac{1}{2}\mathbf{x}^T\mathbf{H}\mathbf{x}$, which is what any function looks like near a critical point. None of these matrices is diagonal - the answer is in the eigenvalues, not the entries.

Further Practice

- Budgetary constraints
- Lagrange multipliers, examples
- Interpretation of Lagrange multipliers

Part III: Principal Component Analysis (PCA)

Principal Components Analysis

PCA is a dimension reduction technique that allows you to represent your data in less space while preserving as much information as possible. It therefore has a variety of applications in:

- File compression
- Genomics
- Facial recognition and computer vision
- Finance and economics
- Climate science
- Social science

Laszlo the cat

This week's lab applies PCA to this image.



Cats and the Internet (wikipedia)

Motivation

Suppose I have an $n \times p$ data matrix $\mathbf{X}$. You want to save my whole matrix to take it with you, but I'm only letting you take one column.

n observations, p variables:

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}$$

Which Column Do You Keep?

$$\mathbf{X} = \begin{bmatrix} \textbf{Student?} & \textbf{Country} & \cdots \\ 1 & \text{India} & \cdots \\ 1 & \text{Germany} & \cdots \\ \vdots & \vdots & \ddots \\ 1 & \text{USA} & \cdots \end{bmatrix}$$

Note that the first column is the same for everybody. It has no variance, and a column with no variance tells you nothing about who is who.

That is the general idea: keep the column that varies most.

Can We Do Better?

I said one column. I never said it had to be a column of $\mathbf{X}$.

Consider instead a column that is a linear combination of the columns of $\mathbf{X}$:

$$\mathbf{z}_1 = \phi_{11}\mathbf{x}_1 + \phi_{21}\mathbf{x}_2 + \cdots + \phi_{p1}\mathbf{x}_p = \mathbf{X}\boldsymbol{\phi}_1$$

How do you choose $\boldsymbol{\phi}_1$?

$\mathbf{X} \in \mathbb{R}^{n \times p}, \quad \boldsymbol{\phi} \in \mathbb{R}^{p}, \quad \mathbf{z} \in \mathbb{R}^{n}$

$\boldsymbol{\phi}$ is called the loading.

$\mathbf{z}$ is called the score.

Most Information = Most Variance

Choose the loadings (i.e. the ϕ_{j1} s) that maximise the variance of $\mathbf{z}_1$.

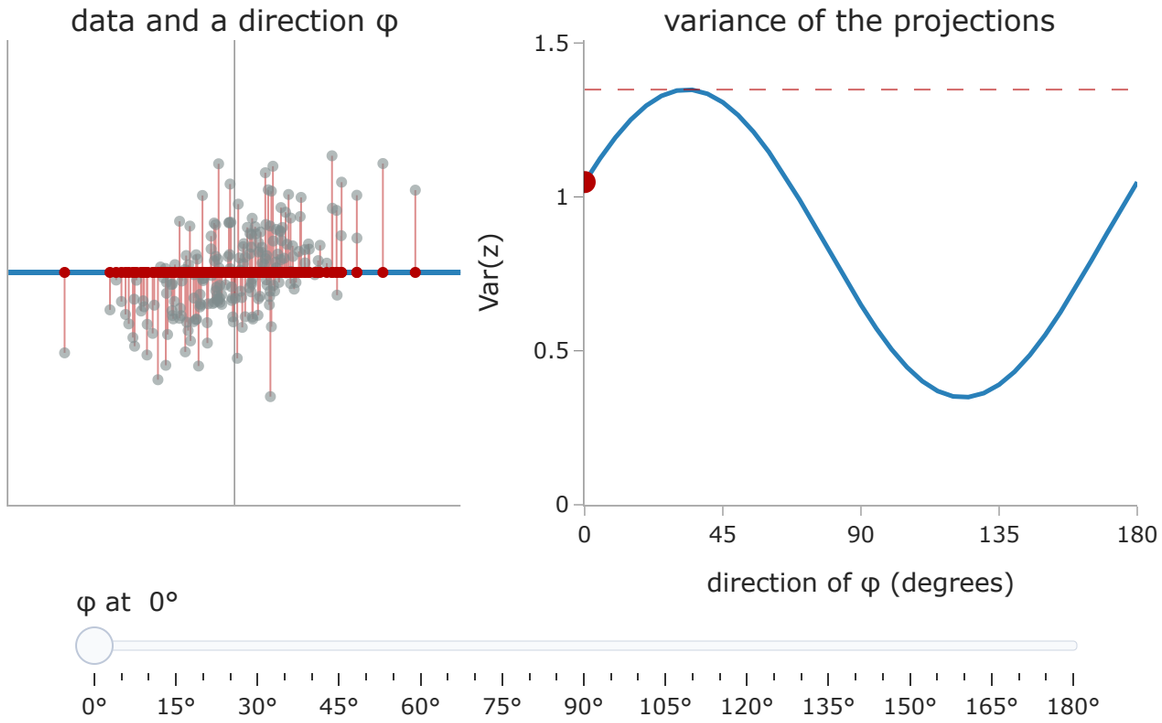
Assume the columns of $\mathbf{X}$ are centred, so $\bar{z}_1 = 0$. Then you would solve the maximization problem:

$$\max_{\boldsymbol{\phi}_1} \ \mathrm{Var}(\mathbf{z}_1) = \max_{\boldsymbol{\phi}_1} \ \frac{1}{n}\sum_{i=1}^{n}(z_{i1} - \bar{z}_1)^2 = \max_{\boldsymbol{\phi}_1} \ \frac{1}{n}\sum_{i=1}^{n} z_{i1}^2 = \max_{\boldsymbol{\phi}_1} \ \frac{1}{n}\mathbf{z}_1^T\mathbf{z}_1$$

subject to the constraint

$$\sum_{j=1}^{p} \phi_{j1}^2 = \boldsymbol{\phi}_1^T\boldsymbol{\phi}_1 = 1$$

The variance of z as a function of ϕ



Finding the PCA with Matrix Calculus

Form the Lagrangian

$$\begin{aligned}\mathcal{L} &= \frac{1}{n}\mathbf{z}_1^T\mathbf{z}_1 - \lambda\left(\boldsymbol{\phi}_1^T\boldsymbol{\phi}_1 - 1\right) \\ &= \frac{1}{n}(\mathbf{X}\boldsymbol{\phi}_1)^T\mathbf{X}\boldsymbol{\phi}_1 - \lambda\left(\boldsymbol{\phi}_1^T\boldsymbol{\phi}_1 - 1\right) \\ &= \frac{1}{n}\boldsymbol{\phi}_1^T\mathbf{X}^T\mathbf{X}\boldsymbol{\phi}_1 - \lambda\left(\boldsymbol{\phi}_1^T\boldsymbol{\phi}_1 - 1\right)\end{aligned}$$

Take the gradient and set it equal to the zero vector:

$$\nabla_{\boldsymbol{\phi}_1}\mathcal{L} = \frac{2}{n}\left(\mathbf{X}^T\mathbf{X}\right)\boldsymbol{\phi}_1 - 2\lambda\boldsymbol{\phi}_1 = \mathbf{0}$$

Rearrange and solve:

$$\frac{1}{n}\mathbf{X}^T\mathbf{X}\,\boldsymbol{\phi}_1 = \lambda\boldsymbol{\phi}_1$$

Does that remind you of something?

Optimization as Eigendecomposition

Write $\mathbf{S} = \frac{1}{n}\mathbf{X}^T\mathbf{X}$:

$$\mathbf{S}\boldsymbol{\phi}_1 = \lambda\boldsymbol{\phi}_1$$

The solution to our optimization problem is an eigendecomposition on some matrix $\mathbf{S}$.

And $\mathbf{S}$ is an important matrix: it is the variance-covariance matrix of our data matrix, $\mathbf{X}$.

$\boldsymbol{\phi}_1$ is an eigenvector of $\mathbf{S}$, and λ its eigenvalue.

The Variance-Covariance Matrix

With the columns of $\mathbf{X}$ centred at 0:

$$\mathbf{S} = \text{Cov}(\mathbf{X}) = \frac{1}{n}\mathbf{X}^T\mathbf{X}$$

$$= \frac{1}{n}\begin{bmatrix} \sum_i x_{i1}^2 & \sum_i x_{i1}x_{i2} & \cdots & \sum_i x_{i1}x_{ip} \\ \sum_i x_{i2}x_{i1} & \sum_i x_{i2}^2 & \cdots & \sum_i x_{i2}x_{ip} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_i x_{ip}x_{i1} & \sum_i x_{ip}x_{i2} & \cdots & \sum_i x_{ip}^2 \end{bmatrix}$$

Variances down the diagonal, covariances off-diagonal.

It is symmetric, and $p \times p$. Its size has nothing to do with how many observations you have.

Substitute $\mathbf{S}\boldsymbol{\phi}_1 = \lambda\boldsymbol{\phi}_1$ back into the objective:

$$\text{Var}(\mathbf{z}_1) = \boldsymbol{\phi}_1^T\mathbf{S}\boldsymbol{\phi}_1 = \boldsymbol{\phi}_1^T(\lambda\boldsymbol{\phi}_1) = \lambda\,\boldsymbol{\phi}_1^T\boldsymbol{\phi}_1 = \lambda$$

The variance we are maximising is the eigenvalue. So take the eigenvector belonging to the largest one.

In Summary

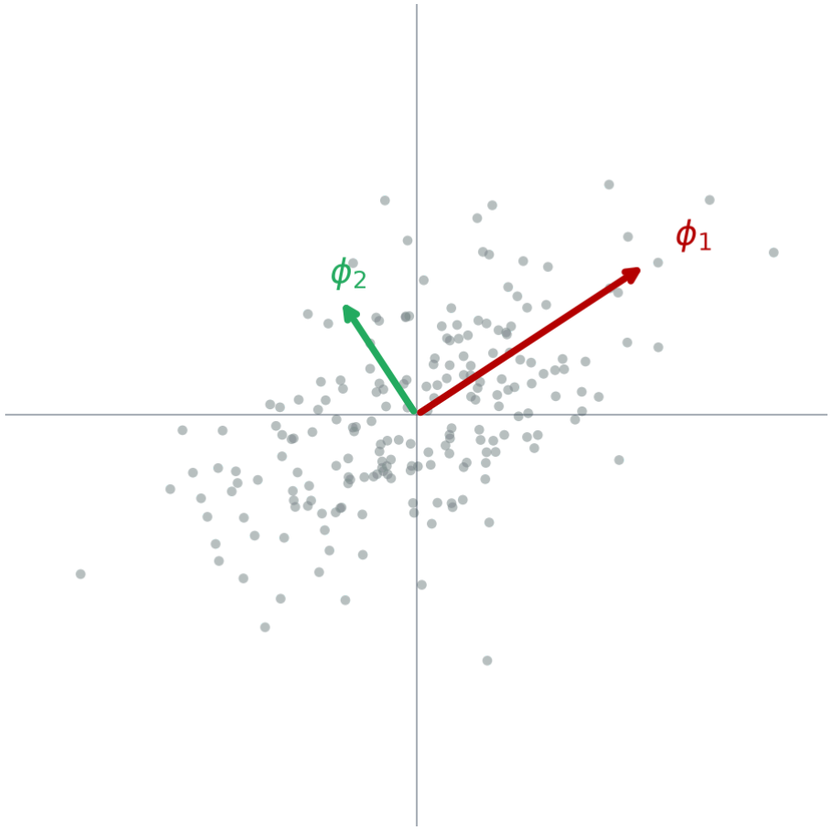
The first principal component is the column you would take away if you could only take one column of $\mathbf{X}$ with you.

- But it was formed brilliantly: as a linear combination of all the columns of $\mathbf{X}$.
- Its loadings $\boldsymbol{\phi}_1$ are the ones that maximise its variance.
- And one can equivalently get there by taking the variance-covariance matrix of $\mathbf{X}$, performing eigendecomposition on it, and taking the eigenvector corresponding to the largest eigenvalue.
- We can interpret this first principal component as a projection of the $n \times p$ matrix onto one dimension (i.e., into single n-length vector).

The Second Principal Component

Now suppose I let you have another column, $\mathbf{z}_2$. Which column will you choose next?

- You should choose the linear combination of the columns of $\mathbf{X}$, $\mathbf{x}_1, \ldots, \mathbf{x}_p$, that has maximal variance out of all the linear combinations that are uncorrelated with the first principal component $\mathbf{z}_1$
- This amounts to the same maximization problem with a new constraint
- It is also the same as taking the eigenvector corresponding to the second-largest eigenvalue of the variance-covariance matrix
- The next principal components are constructed similarly…



PCA and SVD

SVD factored any matrix as $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$.

Put that into $\mathbf{S}$:

$$\mathbf{S} = \frac{1}{n}\mathbf{X}^T\mathbf{X} = \frac{1}{n}\mathbf{V}\mathbf{\Sigma}^T\mathbf{U}^T\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T = \mathbf{V}\left(\frac{\mathbf{\Sigma}^2}{n}\right)\mathbf{V}^T$$

using $\mathbf{U}^T\mathbf{U} = \mathbf{I}$.

That is an eigendecomposition of $\mathbf{S}$, so we can read off the principal components of $\mathbf{X}$:

- the loadings $\boldsymbol{\phi}_k$ are the columns of $\mathbf{V}$, i.e. the right singular vectors
- the scores $\mathbf{Z} = \mathbf{X}\mathbf{V} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T\mathbf{V} = \mathbf{U}\mathbf{\Sigma}$ are the columns of $\mathbf{U}$ scaled by the corresponding singular values.
- the variances λ_k are σ_k^2/n

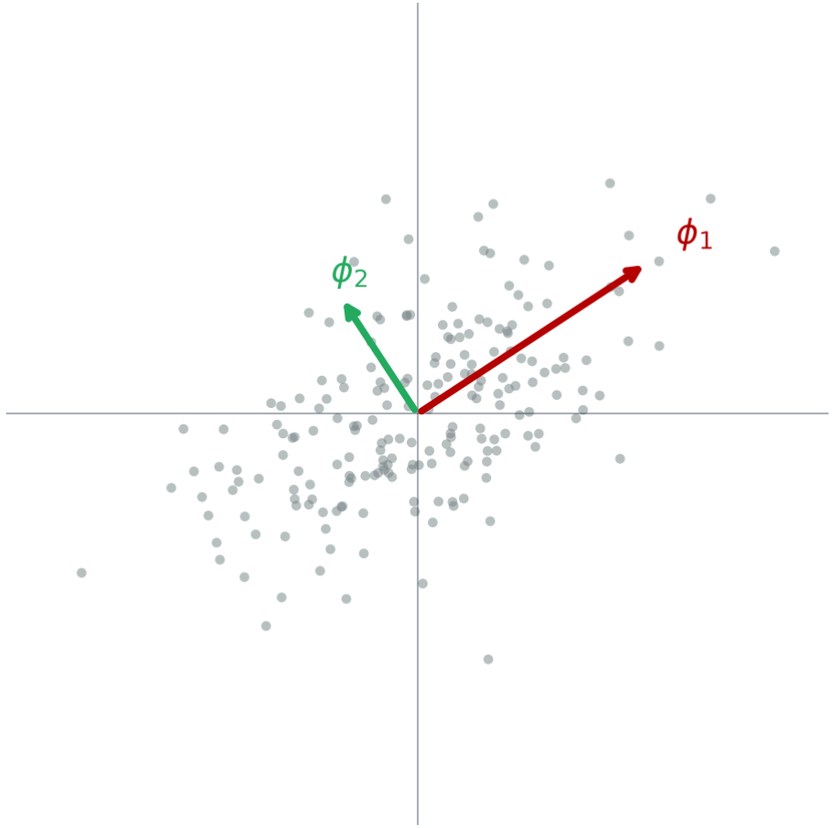
$\mathbf{X} \in \mathbb{R}^{n\times p}, \quad \boldsymbol{\phi}_k \in \mathbb{R}^p, \quad \mathbf{z}_k \in \mathbb{R}^n$

$\mathbf{U} \in \mathbb{R}^{n\times n}, \quad \mathbf{V} \in \mathbb{R}^{p\times p}, \quad \mathbf{\Sigma} \in \mathbb{R}^{n\times p}, \quad \mathbf{\Sigma}^T\mathbf{\Sigma} = \mathbf{\Sigma}^2 \in \mathbb{R}^{p\times p}$

Reconstructing the Data Component by Component

$$\hat{\mathbf{X}} = \mathbf{z}_1 \boldsymbol{\phi}_1^T + \mathbf{z}_2 \boldsymbol{\phi}_2^T + \cdots + \mathbf{z}_k \boldsymbol{\phi}_k^T$$

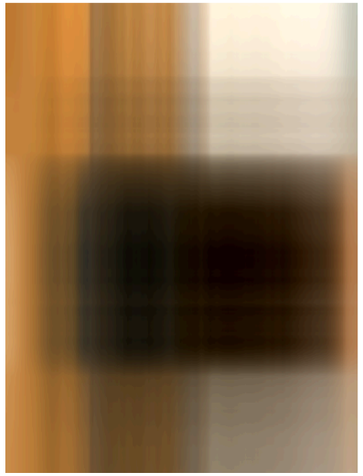
At $k = p$ this is $\mathbf{X}$ exactly. Below that, it is the best rank-k approximation there is.



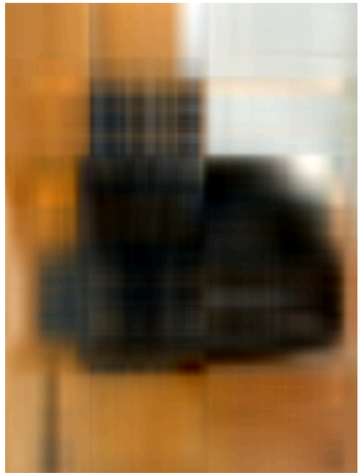
$\mathbf{X} \in \mathbb{R}^{n \times p}, \quad \boldsymbol{\phi}_k \in \mathbb{R}^p, \quad \mathbf{z}_k \in \mathbb{R}^n$

Reconstructing Laszlo the Cat Component by Component

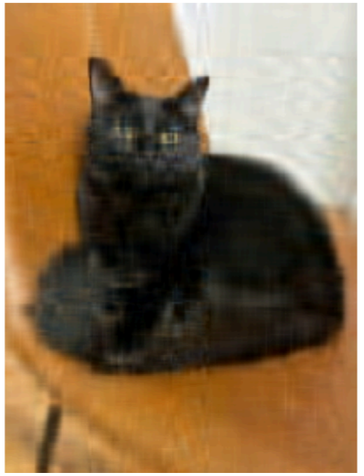
$k = 1$
71.6% of variance



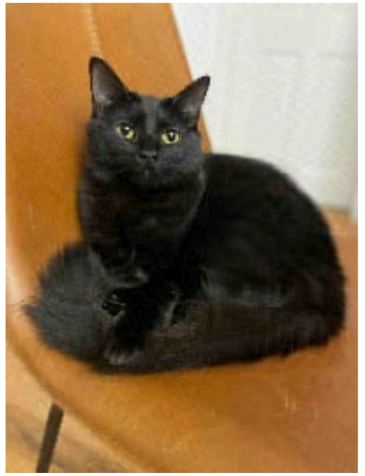
$k = 5$
93.8% of variance



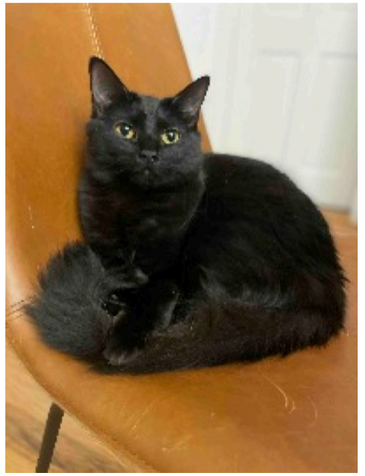
$k = 20$
98.5% of variance



$k = 50$
99.5% of variance



original
200 x 3 columns



Part IV: Further Learning - Optimization Algorithms

Further Learning: Optimization Algorithms

Gradient descent

- Minimizes functions by taking repeated steps in the opposite direction of the (approximate) gradient at the current point, because this is the direction of steepest descent

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \nabla f(\mathbf{x}_t)$$

- The workhorse of machine learning. Its variants, such as SGD, mini-batch, Adam, differ mainly in how they estimate that gradient and how they choose η.

Newton-Raphson

- An iterative method for finding critical points that uses Taylor Series approximation; uses Hessian for faster convergence
- Good for smooth and convex problems

Quasi-Newton Methods (e.g., BFGS)

- Approximates the Hessian for computational efficiency
- Ideal for large-scale optimization problems

Further Learning: Optimization Algorithms

Conjugate gradient

- effective for large, sparse systems
- avoids computing second derivatives directly

Simulated annealing

- a probabilistic technique for global optimization
- helps escape local minima in non-convex problems

Genetic algorithms

- inspired by natural selection
- effective for complex, non-linear problems

Adam (adaptive moment estimation)

- an extension of stochastic gradient descent
- widely used to train deep neural networks

Summary

Concept	Key formula
Lagrangian	$\mathcal{L} = f + \lambda g$
First-order conditions	$\nabla \mathcal{L} = \mathbf{0}$ - includes the constraint
Gradient	$(\nabla_{\mathbf{x}} f)_i = \partial f / \partial x_i$, same shape as $\mathbf{x}$
Hessian	$H_{ij} = \partial^2 f / \partial x_i \partial x_j$; symmetric
Covariance matrix	$\mathbf{S} = \frac{1}{n} \mathbf{X}^T \mathbf{X}$ (centred), $p \times p$, symmetric
PCA problem	$\max_{\boldsymbol{\phi}} \boldsymbol{\phi}^T \mathbf{S} \boldsymbol{\phi}$ s.t. $\boldsymbol{\phi}^T \boldsymbol{\phi} = 1$
PCA solution	$\mathbf{S} \boldsymbol{\phi}_k = \lambda_k \boldsymbol{\phi}_k$; take the largest λ_k
Via the SVD	$\mathbf{X} = \mathbf{U} \boldsymbol{\Sigma} \mathbf{V}^T \Rightarrow \boldsymbol{\phi}_k = \mathbf{v}_k, \lambda_k = \sigma_k^2 / n$
Variance carried	$\mathrm{Var}(\mathbf{z}_k) = \lambda_k$
Reconstruction	$\hat{\mathbf{X}} = \sum_{j \le k} \mathbf{z}_j \boldsymbol{\phi}_j^T$

Reading

Hertzmann, Fleet & Brubaker, *CSC 411 Course Notes* (U of Toronto, 2015)

- Ch. 14, p. 83–91 - Lagrange multipliers, including the circle example worked on p. 84

Kolter & Do, *Linear Algebra Review and Reference*

- Ch. 4, p. 20–26 - matrix calculus: gradients, the Hessian, and gradients of quadratic forms

James et al., *An Introduction to Statistical Learning*

- Ch. 6.3.1, p. 253–256 - principal components regression
- Ch. 12.2, p. 496–508 - PCA in full, including the scree plot
- Ch. 12.5.1 - PCA in practice

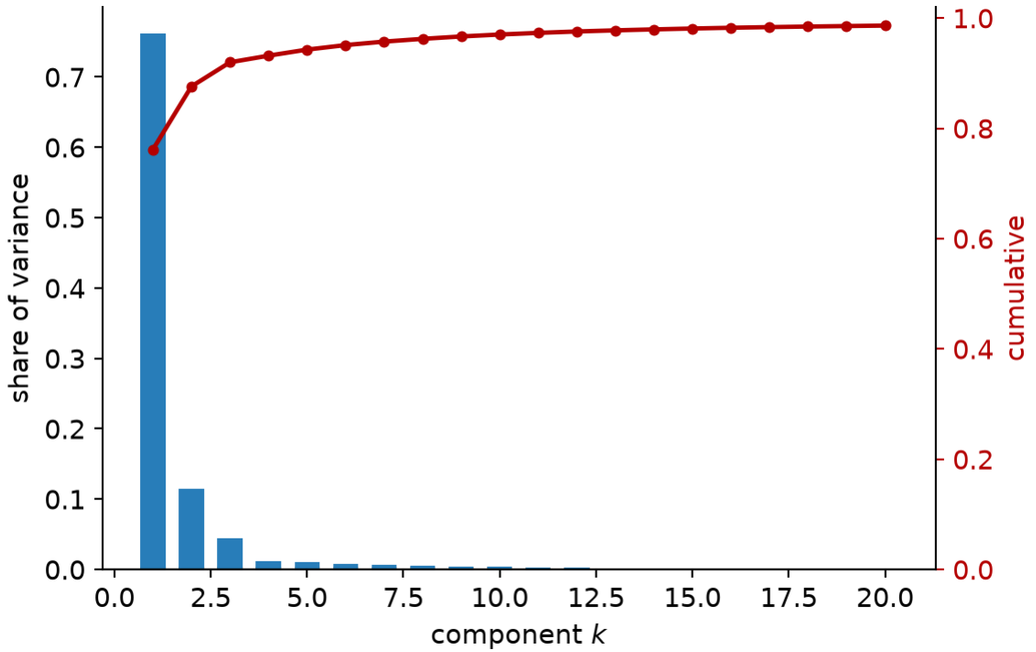
Appendix

The scree plot

Component k carries variance λ_k. Its share of the total is the proportion of variance explained:

$$\text{PVE}_k = \frac{\lambda_k}{\sum_{j=1}^{p} \lambda_j}$$

Plot the shares, and the running total, and look for where the curve flattens - the scree plot.



Notes on centering and scaling

$\mathbf{S} = \frac{1}{n}\mathbf{X}^T\mathbf{X}$ is the covariance matrix only if the columns are centred.

Skip that step and the first component points at the mean of the data rather than at the direction it spreads in - a component that describes where the cloud is, not what is in it.

Whether to also scale each column to unit variance is a real decision:

- variables in different units (kg, years, €) - scale, or the one with the biggest numbers wins by accident
- variables in the same units (pixel intensities, all in $[0, 1]$) - don't: the differences in spread are real information